ISHAAN JAIN

Q1. Distributed Systems with Examples

A distributed system is a collection of independent computers that appear as a single coherent system. These system consist of multiple autonomous computational entities connected through a network that work together to achieve common goals.

KEY CHARACTERISTICS

(1) Heterogeneity - Distributed systems can consist of various hardware and software components from different vendors that work together effectively

(2) Scalability - These systems can grow easily by adding more computers, allowing them to handle increased workloads and accomodate business growth.

(3) Fault Tolerance - If one computer fails, others can continue working, ensuring system reliability and continous operation.

(4) Concurrency - Multiple processes can execute simultaneously across different nodes, improving overall system performance.

(5) Transparency - The complexity of the distributed nature is hidden from users, making the system appear as a single entity

(6) Resource Sharing - Hardware, software and data resources are shared among multiple computers in network.

Examples -

(1) Client - server systems - Web applications where browsers request services from web servers that provide resources and process requests.

(2) Peer - to - Peer Systems - Each nodes act as both client and server, sharing resources directly without central co-ordination. BitTorrent file-sharing networks exemplify this architecture.

(3) Grid Computing System - Multiple computers are connected to form a virtual supercomputer, commonly used for scientific research requiring massive computational power.

(4) Cloud computing System- Provide on-demand access to shared computing resources like servers, storage and applications through services like AWS, Azure and Google Cloud Platform.

Q2. System Models in Distributed Systems

System Models provide frameworks for understanding and designing distributed systems by describing their structure, behaviour and properties.

ARCHITECTURAL MODELS

(1) Layered Architecture- Organizes the system into multiple layers where each layer provides specific services and communicates with adjacent layers using well-defined protocols.

(2) Object-based architecture- Components are organized as objects that interact through method invocations, promoting modularity and reusability.

(3) Event-based Architecture- Components interact by sending and responding to events rather than direct requests, commonly used in IoT systems and real-time applications.

(4) Client-server Model - Centralized approach where clients initiate service requests and server respond by providing those services using protocols like TCP/IP and HTTP.

(5) Microservices Model - Complex applications are decomposed into small, independent services that perform specific functions and communicate over networks.

FUNDAMENTAL MODELS

(1) Interaction Model - Deals with communication details among components, including timing, performance and synchronous versus asynchronous communication patterns.

(2) Failure Model - Describes different types of failures that can occur in distributed systems, including crash failures, omission failures, Byzantine failures.

(3) Security Model - Addresses security concerns including authentication, authorization, data integrity, protection against malicious attacks in distributed environments.

ISHAAN JAIN

Q3. Time and Global States - Logical Clocks

Time Coordination is crucial in distributed systems since there's no global physical clock. Logical clocks provide mechanisms to order events and maintain consistency across distributed nodes.

Lamport's logical Clock :-

lamport clock are event counters that increment with every interaction, providing a partial ordering of events.

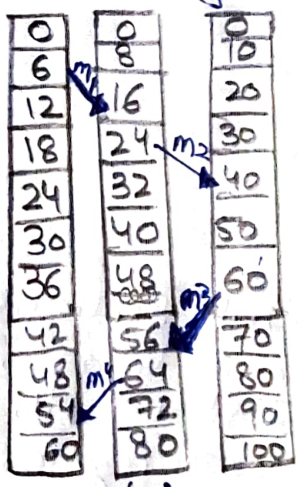
Algorithm :-

- Each process maintains a logical clock LCi
- Before executing an event, increment LCi = LCi + 1
- When sending a message, include current LCi value
- Upon receiving a message with timestamp LCj, update

$$LCi = \max(LCi, LCj) + 1$$

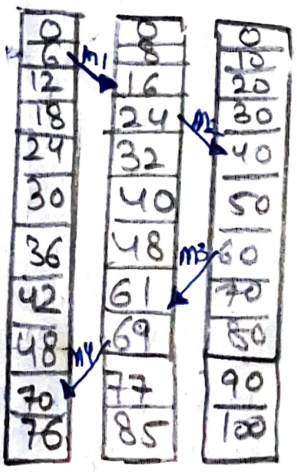
Properties :

lamport clocks ensure that if events a happened-before event b, then LC(a) < LC(b), but the converse isn't necessarily true.



(a)

Three processes each with its own clock. The clock run at different Rates



(b)

lamport's algorithm corrects the clocks

LAMPORT'S ALGORITHM

Vector Clocks-

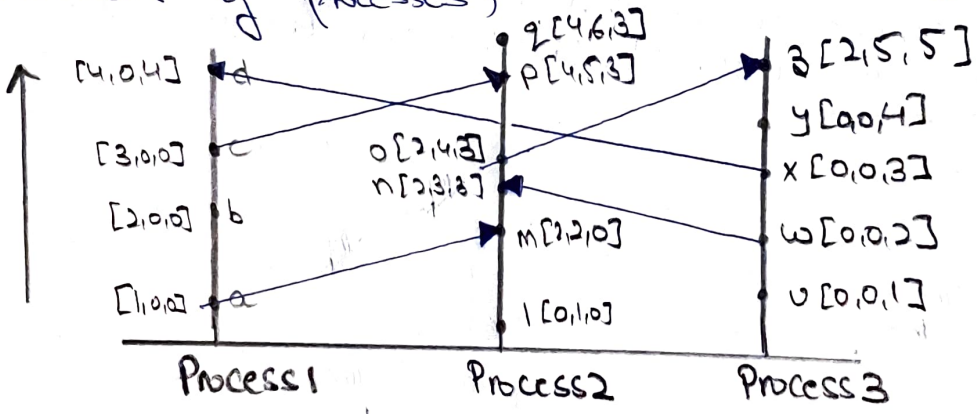
Vector clocks use an array of integers where each element corresponds to a process in the system.

Algorithm-

- Each process Pi maintain vector VC[1.. n] where n is the number of processes.
- Before local event : $VC_i[i] = VC_i[i] + 1$
- When sending message : include current VC_i
- Upon receiving message with VC_j : $VC_i[k] = \max(VC_i[k], VC_j[k])$ for all k, then $VC_i[i] = VC_i[i] + 1$

Advantages : Vector clocks can detect causal relationships and concurrent events reliably, providing stronger ordering guarantees than Lamport clocks.

Limitation- Higher overhead (vector size grows with number of processes)



Q4. Consensus Problem and Distributed Mutual Exclusion

The consensus problem requires multiple processes to agree on a common value or decision despite potential failures. It's fundamental for maintaining consistency in distributed systems.

Challenges in Achieving Consensus

(1) Network Partitions- Communication failures can split the system into isolated groups.

(2) Process failures - Nodes may crash or behave maliciously

(3) Asynchronous Communication - Message delays and ordering issues complicate coordination
(4) Incomplete Information - Processes have limited knowledge of global system state.

Distributed Mutual Exclusion Algorithms

(1) token-based approach - A unique token circulates among processes, only the token holder can access the critical section
(2) Non-token based (Ricart-Agrawala Algorithm) - Uses timestamps and voting mechanisms where process requests permission from all others before entering critical sections.
(3) Quorum-based approach - Process much obtain permission from a majority of nodes before accessing shared resources.
(4) Requirements - All algorithms that must ensure mutual exclusion (only one process in critical section), progress (eventual access), and fairness (bounded waiting) while avoiding deadlocks.
(5) Implementation Challenges - Message passing is the sole communication mechanism, requiring careful handling of message delays and potential failures.